

Corso di laurea in Informatica
Algoritmi 1
A.A. 2025/26

Equazioni di ricorrenza

Tiziana Calamoneri



SAPIENZA
UNIVERSITÀ DI ROMA



Slides realizzate sulla base di quelle preparate da T. Calamoneri e G. Bongiovanni per il corso di Informatica Generale tenuto a distanza nell'A.A. 2019/20

Sommario

Equazioni di ricorrenza

Soluzione delle equazioni di ricorrenza tramite:

- Metodo iterativo
- Metodo dell'albero (con una parentesi sugli alberi binari)
- Metodo di sostituzione
- Metodo principale

Equazioni di ricorrenza (1)

- Valutare il costo computazionale di un algoritmo ricorsivo è, in genere, più laborioso che nel caso degli algoritmi iterativi.
- Infatti, la natura ricorsiva della soluzione algoritmica dà luogo a una funzione matematica di costo anch'essa ricorsiva.
- Trovare la funzione di costo ricorsiva è piuttosto immediato. Essa però deve essere risolta, altrimenti il costo asintotico non può essere quantificato, e questa è la parte meno semplice.
- La funzione matematica ricorsiva che esprime il costo è anche detta ***equazione di ricorrenza***.

Equazioni di ricorrenza (2)

Esempio: costo computaz. della ricerca sequenziale ricorsiva:

```
def Ricerca_seq_ric(A, v, n=len(A)-1):  
    if (A[n]==v):  
        return n  
    if (n==0):  
        return -1  
    else:  
        return Ricerca_seq_ric (A, v, n-1)
```

In generale: $T(n)=\Theta(1)+T(n-1)$

Caso base: $T(0)=\Theta(1)$

Equazioni di ricorrenza (3)

- La parte generale dell'equazione di ricorrenza che definisce $T(n)$ deve essere sempre costituita dalla somma di **almeno due addendi**, di cui **almeno uno contiene la parte ricorsiva** (nell'esempio $T(n-1)$) mentre **uno rappresenta il costo computazionale di tutto ciò che viene eseguito al di fuori della chiamata ricorsiva**.
- ...

Equazioni di ricorrenza (4)

- ...
- Anche se questa parte dovesse essere un solo confronto, il suo costo non può essere ignorato, altrimenti si otterrebbe che la funzione ha una costo computazionale indipendente dalla dimensione del suo input, e questa è data da quanto illustrato nel caso base. Non possiamo dire che ciò sia impossibile, ma è molto improbabile.



Deve sempre essere presente un caso base.

Equazioni di ricorrenza (5)

Esistono alcuni metodi utili per risolvere le equazioni di ricorrenza, che illustreremo:

- metodo iterativo
- metodo di sostituzione
- metodo dell'albero
- metodo principale

Metodo iterativo (1)

Idea:

sviluppare l'equazione di ricorrenza ed esprimerla come somma di termini dipendenti da n e dal caso base

Difficoltà:

maggior quantità di calcoli algebrici rispetto agli altri metodi

Metodo iterativo (2)

Esempio: Equazione relativa alla ricerca seq. ricorsiva:

$$T(n) = T(n - 1) + \Theta(1)$$

$$T(0) = \Theta(1)$$

$$T(n-1) = \Theta(1) + T(n-2)$$

$$T(n) = \Theta(1) + T(n - 1)$$

$$T(n-2) = \Theta(1) + T(n-3)$$

$$T(n) = \Theta(1) + \Theta(1) + T(n - 2)$$

$$T(n-3) = \Theta(1) + T(n-4)$$

$$T(n) = \Theta(1) + \Theta(1) + \Theta(1) + T(n - 3)$$

$$T(n) = \Theta(1) + \Theta(1) + \Theta(1) + \Theta(1) + T(n - 4),$$

ecc. fino a ottenere:

$$T(n) = k \Theta(1) + T(n-k) = \text{(quando } k=n) n \Theta(1) + T(0) = \Theta(n).$$

Metodo iterativo (3)

Esempio: Equazione relativa alla ricerca binaria ricorsiva:

$$T(n) = T(n/2) + \Theta(1) \text{ e } T(1) = \Theta(1)$$

$$T(n) = \Theta(1) + T(n/2)$$

$$T(n) = \Theta(1) + \Theta(1) + T(n/2^2)$$

$$T(n) = \Theta(1) + \Theta(1) + \Theta(1) + T(n/2^3)$$

...

$$T(n) = T(n/2^k) + k \Theta(1)$$

Quando $k = \log n$ si ha $n/2^k = 1$ quindi, per tale valore di k , ci fermiamo ed otteniamo:

$$T(n) = \Theta(1) + \log n \Theta(1) = \Theta(\log n).$$

$$T(n/2) = \Theta(1) + T(n/2^2)$$

$$T(n/2^2) = \Theta(1) + T(n/2^3)$$

Metodo iterativo (4)

Esempio: Possiamo riscrivere un algoritmo di ricerca sequenziale ricorsiva diverso dal precedente:

```
def Ric_seq_ric2(A, v, i_min=0, i_max=len(A)-1):  
    if (i_max < i_min): return FALSE  
    m=(i_max+i_min)//2  
    if (A[m]==v): return TRUE  
    else return (Ricerca_seq_ric2 (A, v, i_min, m-1)  
                OR Ricerca_seq_ric2 (A, v, m+1, i_max))
```

costo: $T(n) = 2T(n/2) + \Theta(1)$
 $T(0) = \Theta(1)$

Metodo iterativo (5)

Esempio: Equazione relativa alla ricerca sequenziale
ricorsiva (seconda versione):

$$T(n) = 2T(n/2) + \Theta(1) \text{ e } T(0) = \Theta(1)$$

$$T(n/2) = \Theta(1) + 2T(n/2^2)$$

$$T(n) = \Theta(1) + 2 T(n/2)$$

$$\begin{aligned} T(n) &= \Theta(1) + 2[\Theta(1) + 2T(n/2^2)] = \\ &= \Theta(1) + 2 \Theta(1) + 2^2 T(n/2^2) = \end{aligned}$$

$$\dots = 2^k T(n/2^k) + \sum_{i=0..k-1} 2^i \Theta(1) = \dots$$

... proseguo finché l'argomento della funzione raggiunga
il caso base: $n/2^k=0$. Questa condizione non produce
risultati... il problema è il caso base...

Metodo iterativo (6)

Calcoliamo $T(1)$:

```
def Ric_seq_ric2(A, v, i_min=0, i_max=len(A)-1):  
    if (i_max < i_min): return FALSE  
    m=(i_max+i_min)//2  
    if (A[m]==v): return TRUE  
    else return (Ricerca_seq_ric2 (A, v, i_min, m-1)  
                OR Ricerca_seq_ric2 (A, v, m+1, i_max))
```

$T(1)=\Theta(1)$ perciò:

$$\dots = 2^k T(n/2^k) + \sum_{i=0..k-1} 2^i \Theta(1) = \dots$$

$$(2^{\log n} - 1)/(2 - 1)$$

... proseguo finché $n/2^k=1$ cioè $k=\log n$...

$$\begin{aligned} \dots &= 2^{\log n} \Theta(1) + \sum_{i=0.. \log n - 1} 2^i \Theta(1) = \\ &= \Theta(n) \end{aligned}$$

Metodo iterativo (7)

A volte le cose sono un po' più complicate...

Esempio: Equazione relativa al calcolo dell'n-esimo numero di

Fibonacci: $T(n) = T(n-1) + T(n-2) + \Theta(1)$

$$T(0)=T(1) = \Theta(1)$$



Provando ad usare il metodo iterativo:

$$\begin{aligned} T(n) &= T(n-1) + T(n-2) + \Theta(1) = T(n-2) + 2T(n-3) + T(n-4) + 3\Theta(1) = \\ &= T(n-3) + 3T(n-4) + 3T(n-5) + T(n-6) + 6\Theta(1) = \dots \end{aligned}$$

Quando non riusciamo ad ottenere un'espressione che sappiamo gestire: maggiorazioni e minorazioni!

Metodo iterativo (8)

Segue Esempio: Equazione numeri di Fibonacci:

- $T(n) = T(n-1) + T(n-2) + \Theta(1)$
- $T(0)=T(1) = \Theta(1)$

A partire da $T(n)=T(n-1)+T(n-2) + \Theta(1)$ e
da $T(n-1) \geq T(n-2)$:

- $T(n) \leq 2T(n-1) + \Theta(1)$: questa disuguaglianza ci permetterà di derivare un limite superiore O ;
- $T(n) \geq 2T(n-2) + \Theta(1)$: questa disuguaglianza ci permetterà di derivare un limite inferiore Ω .

Metodo iterativo (9)

Segue Esempio: Equazione numeri di Fibonacci:

- $T(n) = T(n-1) + T(n-2) + \Theta(1)$
- $T(0)=T(1) = \Theta(1)$

$$\begin{aligned} T(n) &\leq 2T(n-1) + \Theta(1) \leq 2^2T(n-2) + 2^1\Theta(1) + \Theta(1) \leq \\ &\leq 2^3T(n-3) + 2^2\Theta(1) + 2^1\Theta(1) + \Theta(1) \leq \dots \\ &\leq 2^kT(n-k) + \sum_{i=0}^{k-1} 2^i \Theta(1) \end{aligned}$$

Il procedimento si ferma quando $n-k = 1$, ossia
 $k = n-1$ per cui otteniamo:

$$\begin{aligned} T(n) &\leq 2^{n-1} \Theta(1) + \sum_{i=0}^{n-2} 2^i \Theta(1) = 2^{n-1} \Theta(1) + (2^{n-1}-1)\Theta(1)= \\ &= (2^n-1)\Theta(1) \end{aligned}$$

e dunque troviamo che **$T(n) = O(2^n)$** .

Metodo iterativo (10)

Segue Esempio: Equazione numeri di Fibonacci:

- $T(n) = T(n-1) + T(n-2) + \Theta(1)$
- $T(0)=T(1) = \Theta(1)$

$$\begin{aligned} T(n) &\geq 2T(n-2) + \Theta(1) \geq 2^2T(n-4) + 2^1\Theta(1) + \Theta(1) \geq \\ &\geq 2^3T(n-6) + 2^2\Theta(1) + 2^1\Theta(1) + \Theta(1) \geq \dots \geq 2^kT(n-2k) + \sum_{i=0}^{k-1} 2^i \Theta(1) \end{aligned}$$

Il procedimento si ferma quando $n-2k = 0$

(se n è pari; per n dispari il procedimento differisce per alcuni dettagli, ma il comportamento asintotico non cambia),

ossia $k = n/2$ per cui otteniamo:

$$\begin{aligned} T(n) &\geq 2^{n/2} \Theta(1) + \sum_{i=0}^{\frac{n}{2}-1} 2^i \Theta(1) = 2^{n/2} \Theta(1) + (2^{n/2} - 1)\Theta(1) = \\ &(2^{n/2} - 1)\Theta(1) \text{ da cui } \mathbf{T(n) = \Omega(2^{n/2})}. \end{aligned}$$

Metodo iterativo (12)

Segue Esempio: Equazione numeri di Fibonacchi:

- $T(n) = T(n-1) + T(n-2) + \Theta(1)$
- $T(0)=T(1) = \Theta(1)$

Anche se non siamo in grado di trovare una funzione asintotica precisa (Θ), possiamo comunque concludere che il calcolo dei numeri di Fibonacci con una tecnica ricorsiva richiede un tempo esponenziale in n , visto che

$$k_1 2^{n/2} \leq T(n) \leq k_2 2^n$$

per opportune costanti k_1 e k_2 .

Metodo dell'albero (1)

Idea:

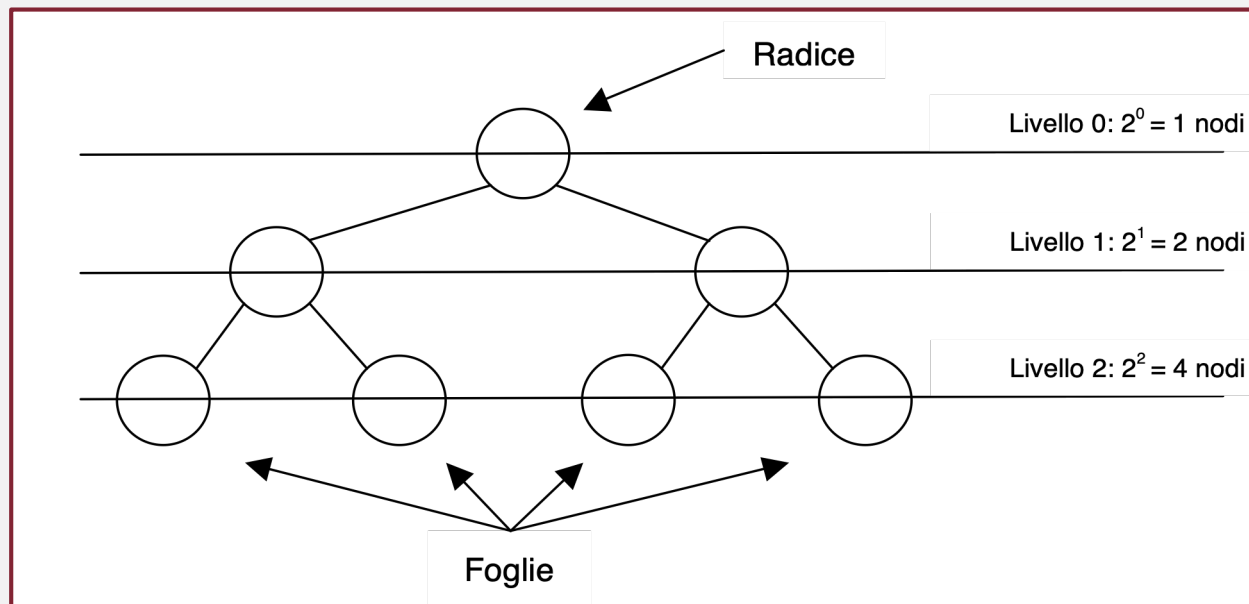
rappresentare graficamente lo sviluppo del costo computazionale dell'algoritmo, in modo da poterla valutare con maggior facilità rispetto al metodo iterativo

Difficoltà:

come il metodo iterativo

Una parentesi: Alberi binari (1)

In un albero binario completo di altezza h:



- il numero delle foglie è 2^h
- il numero dei nodi interni è

$$\sum_{i=0}^{h-1} 2^i = \frac{2^h - 1}{2 - 1} = 2^h - 1$$

- il numero totale dei nodi è $2^h + 2^h - 1 = 2^{h+1} - 1$.

Una parentesi: Alberi binari (2)

Di conseguenza, l'altezza h di un albero binario completo è calcolabile come segue:

Il numero totale dei nodi è:

$$n = 2^{h+1} - 1$$

da cui segue che:

$$\log(n+1) = h+1$$

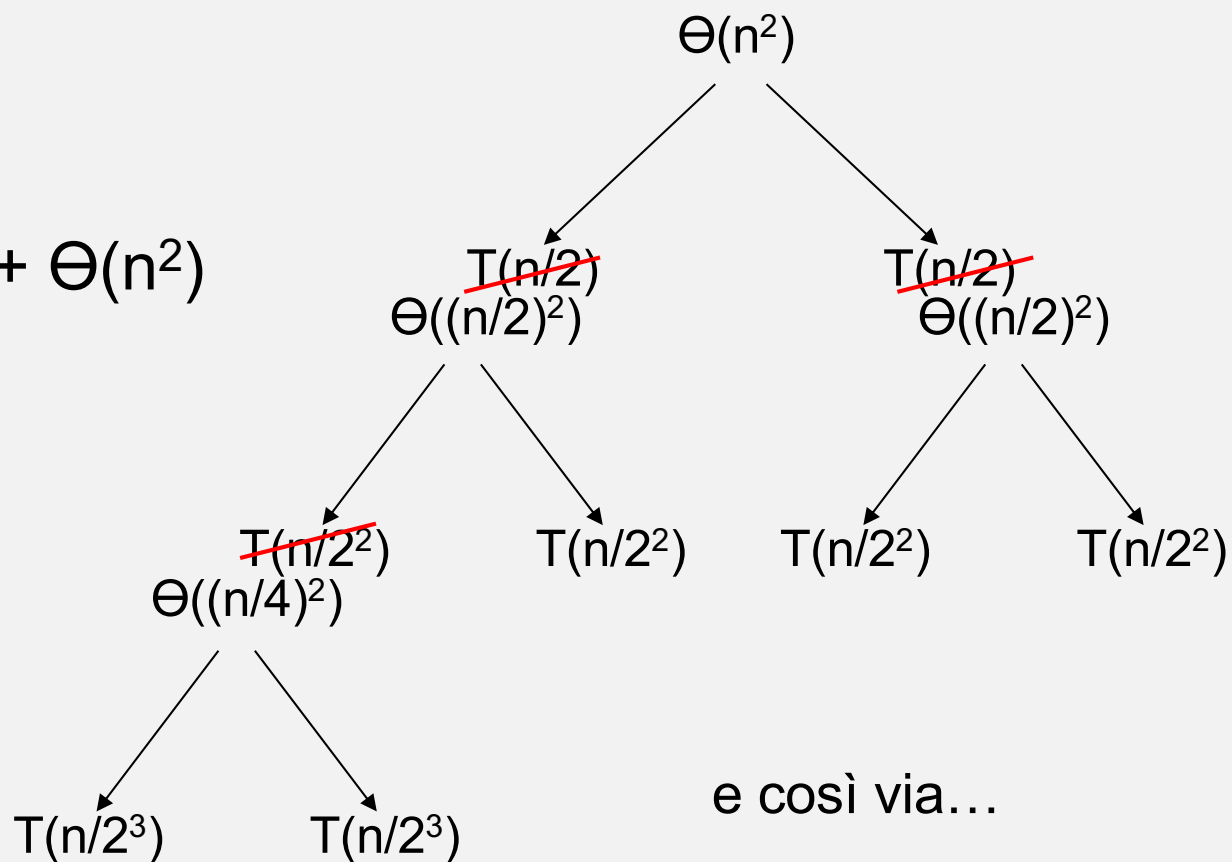
e quindi:

$$h = \log(n+1) - 1 = \log((n+1)/2)$$

Metodo dell'albero (2)

Esempio:

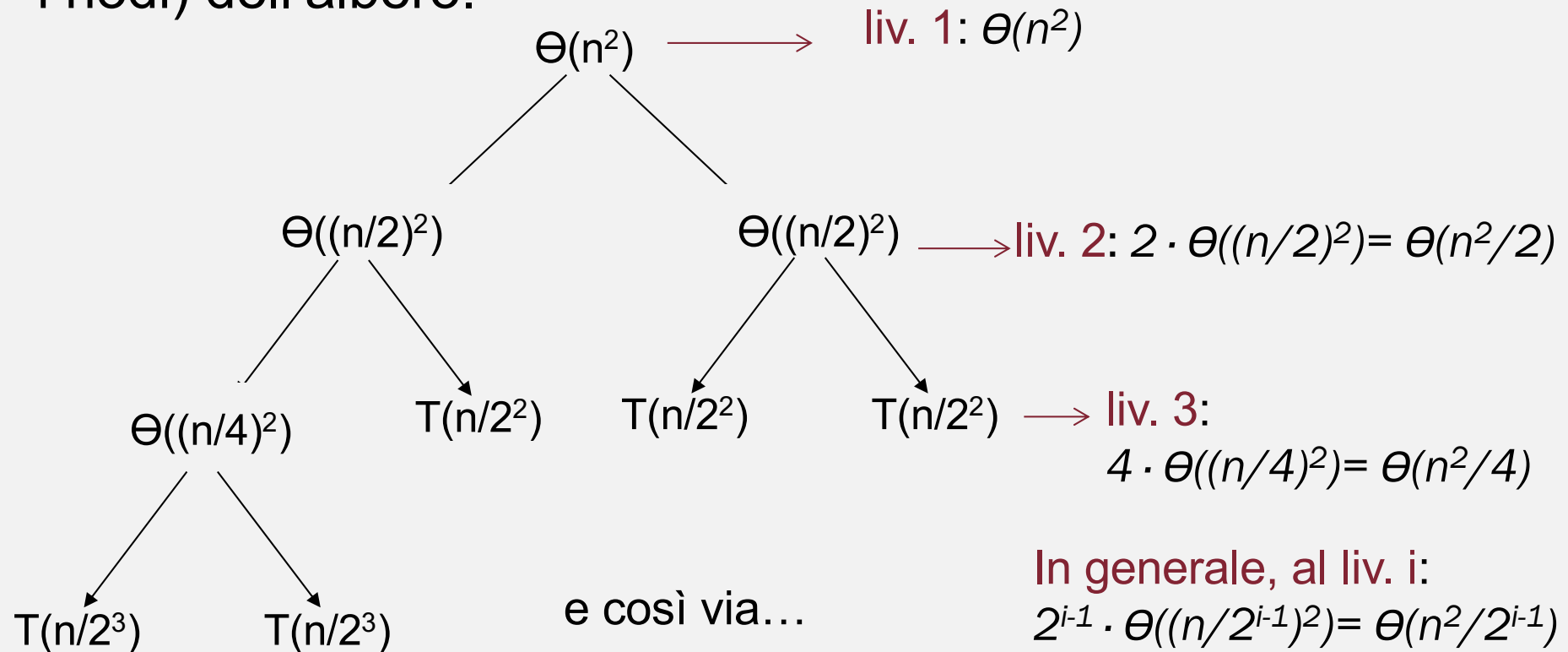
- $T(n) = 2T(n/2) + \Theta(n^2)$
- $T(1) = \Theta(1)$



Metodo dell'albero (3)

segue Esempio:

Una volta completato l'albero, il costo è dato dalla somma dei contributi di tutti i livelli (cioè le “righe” in cui sono disposti i nodi) dell'albero.



Metodo dell'albero (4)

segue Esempio:

In questo caso il numero di livelli dell'albero ha un valore tale che $n/2^{i-1}=1$, ossia $i-1=\log n$ da cui $i=\log n+1$.

Sommiamo i contributi di tutti i livelli:

$$\sum_{i=1}^{\log n+1} \Theta\left(\frac{n^2}{2^{i-1}}\right) = n^2 \sum_{j=0}^{\log n} \Theta\left(\frac{1}{2^j}\right) = \Theta(n^2)$$

Metodo dell'albero (5)

Il metodo dell'albero non si comporta SEMPRE in modo identico al metodo iterativo: ci sono equazioni che possono essere risolte in modo più preciso con il metodo dell'albero.

Esempio:

- $T(n) = T(n/3) + T(2n/3) + \Theta(n)$
- $T(1) = \Theta(1)$

E' facile vedere che con il metodo iterativo non si riesce a trovare una soluzione, quindi usiamo la disuguaglianza $T(n/3) \leq T(2n/3)$ per maggiorare e minorare...

Metodo dell'albero (6)

segue Esempio:

Otteniamo le due equazioni di ricorrenza:

$T(n) \leq 2T(2n/3) + \Theta(n)$ e $T(n) \geq 2T(n/3) + \Theta(n)$, entrambe con $T(1) = \Theta(1)$, le cui soluzioni sono:

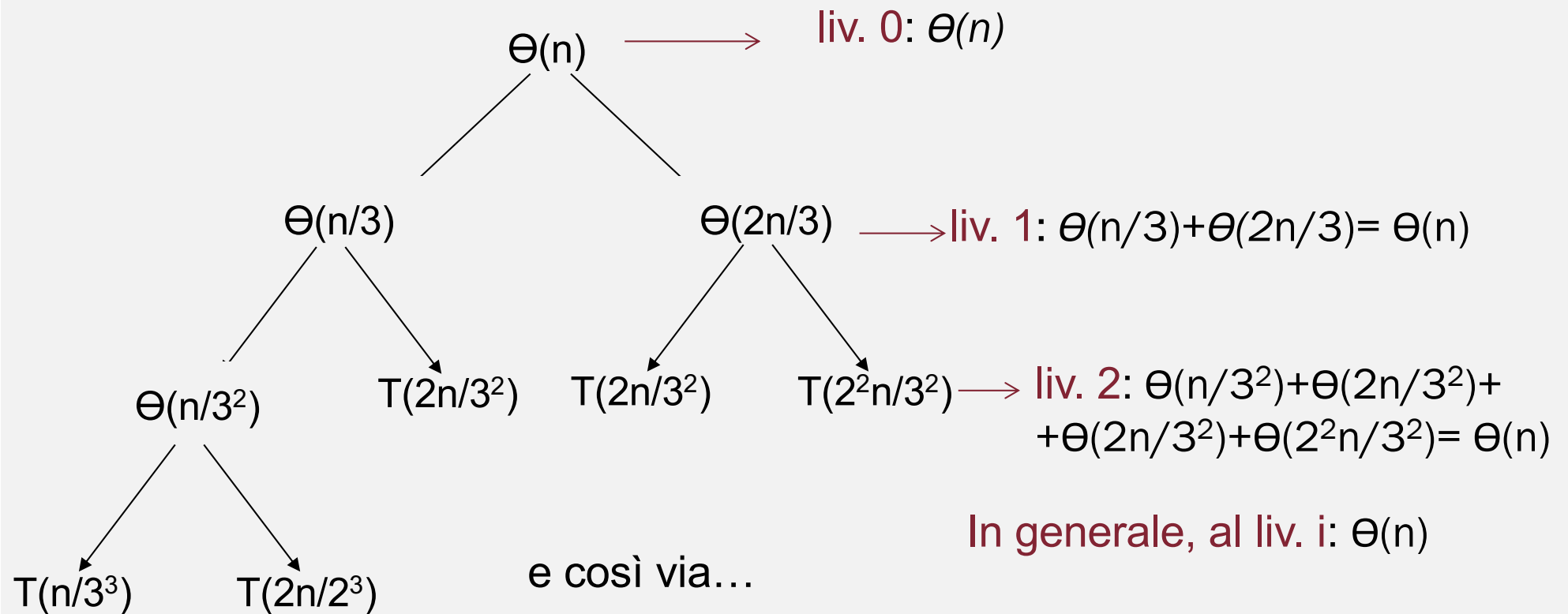
$$T(n) \leq \Theta\left(n^{\log_{3/2} 2}\right) \sim 1,7 \text{ e } T(n) \geq \Theta(n).$$

cioè $T(n) = O\left(n^{\log_{3/2} 2}\right)$ e $T(n) = \Omega(n)$.

Proviamo ora a risolvere con il metodo dell'albero:

Metodo dell'albero (7)

segue Esempio:



Metodo dell'albero (8)

segue Esempio:

La particolarità di questo albero è che non è completo: il ramo più a sinistra ha altezza $\log_3 n$ ed il ramo più a destra ha altezza $\log_{3/2} n$.

Possiamo ancora maggiorare e minorare (assumendo che l'albero sia completo con $\log_3 n$ e $\log_{3/2} n$ livelli, rispettivamente) ottenendo:

$$\log_3 n \cdot \Theta(n) \leq T(n) \leq \log_{\frac{3}{2}} n \cdot \Theta(n)$$

da cui $\Theta(n \log n) \leq T(n) \leq \Theta(n \log n)$ che è un risultato esatto.

Metodo di sostituzione (1)

Idea:

- si ipotizza una soluzione per l'equazione di ricorrenza data;
- si verifica (dimostrando per induzione) se essa “funziona”.

Difficoltà:

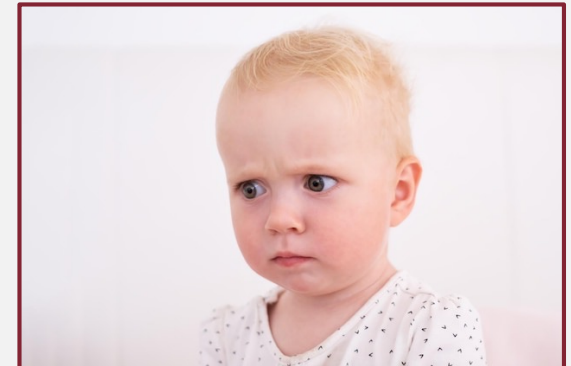
- si deve trovare la funzione più vicina alla vera soluzione, perché tutte le funzioni più grandi (se stiamo cercando O) o più piccole (se stiamo cercando Ω) funzionano.
- In effetti questo metodo serve soprattutto nelle dimostrazioni mentre si sconsiglia di utilizzarlo nella pratica.

Metodo di sostituzione (2)

Esempio: Equazione relativa alla ricerca seq. ricorsiva

$$\begin{aligned} - T(n) &= T(n - 1) + \Theta(1) \\ - T(1) &= \Theta(1) \end{aligned}$$

- Ipotizziamo la soluzione $T(n)=cn$ per una costante opportuna c .
- $T(n)=cn=c(n-1)+\Theta(1)$, cioè $c=\Theta(1)$;
- $T(1)=c=\Theta(1)$.



Quello che abbiamo ottenuto è vero? dobbiamo ricordare che i $\Theta(1)$ provengono da delle costanti ben precise...

Metodo di sostituzione (3)

segue **Esempio**: Equazione relativa alla ricerca seq. ricorsiva

- $T(n) = T(n - 1) + \Theta(1)$
- $T(1) = \Theta(1)$

Per essere certi che le uguaglianze ottenute siano verificate, è necessario **eliminare la notazione asintotica dall'equazione di ricorrenza**, così che le costanti non rimangano "nascoste" nella notazione, conducendo a risultati errati:

- $T(n) = T(n - 1) + c$
- $T(1) = d$

per due costanti c e d fissate.

Metodo di sostituzione (4)

segue **Esempio**: Equazione relativa alla ricerca seq. ricorsiva

- $T(n) = T(n - 1) + c$
- $T(1) = d$ per due costanti c e d fissate.

- Ipotizziamo la soluzione $T(n)=O(n)$, ossia $T(n) \leq kn$ dove k è una costante che va ancora determinata.
- **N.B.** in generale non si può sperare in una soluzione esatta, ma possiamo solo maggiorare o minorare.

Metodo di sostituzione (5)

segue **Esempio**: Equazione relativa alla ricerca seq. ricorsiva

- $T(n) = T(n - 1) + c$
- $T(1) = d$ per due costanti c e d fissate.

- Sostituiamo nel caso base:

$T(1) \leq k$; poiché sapevamo che $T(1)=d$, la disuguaglianza è soddisfatta se e solo se $k \geq d$.

- Sostituiamo nella formulazione ricorsiva:

$T(n) \leq k(n-1)+c = kn-k+c \leq kn$; vera se e solo se $k \geq c$.

La soluzione $T(n) \leq kn$ è corretta per tutti i valori di k tali che $k \geq c$ e $k \geq d$. Poiché un tale k esiste sempre, una volta fissati c e d , $T(n)$ è in $O(n)$.

Metodo di sostituzione (6)

segue **Esempio**: Equazione relativa alla ricerca seq. ricorsiva

- $T(n) = T(n - 1) + c$
- $T(1) = d$ per due costanti c e d fissate.

Che cosa sarebbe successo se avessimo ipotizzato

$$T(n) \leq kn^2?$$

Anche questa soluzione è corretta per tutti i valori di k tali che $k \geq c$ e $k \geq d$.

A noi interessa stimare $T(n)$ asintoticamente tramite la funzione più piccola, e questo è spesso un obiettivo difficile.

Metodo di sostituzione (7)

segue **Esempio**: Equazione relativa alla ricerca seq. ricorsiva

- $T(n) = T(n - 1) + c$
- $T(1) = d$ per due costanti c e d fissate.

Per rendere il nostro risultato stretto, ipotizziamo ora la soluzione $T(n) = \Omega(n)$, ossia $T(n) \geq hn$ dove h è una costante che va ancora determinata.

In modo analogo al caso O , sostituiamo nel caso base ottenendo $h \leq d$.

Sostituiamo nella formulazione ricorsiva dell'equazione di ricorrenza ottenendo:

$$T(n) \geq h(n-1) + c = hn - h + c \geq hn$$

vera se e solo se $h \leq c$.



...

Metodo di sostituzione (8)

segue **Esempio**: Equazione relativa alla ricerca seq. ricorsiva

- $T(n) = T(n - 1) + c$
- $T(1) = d$ per due costanti c e d fissate.

...

Deduciamo dunque che $T(n)$ è un $\Omega(n)$ poiché è possibile trovare un valore h ($h \leq c$, $h \leq d$) per il quale si ha $T(n) \geq hn$.

Dalle due soluzioni: $T(n) = O(n)$ e $T(n) = \Omega(n)$, si ottiene ovviamente che:

$$T(n) = \Theta(n).$$

Metodo di sostituzione (9)

Esempio.

- $T(n)=2T(n/2)+\Theta(1)$
- $T(1)=\Theta(1)$

Dobbiamo eliminare la notazione asintotica:

- $T(n)=2T(n/2)+c$ per due costanti positive fissate c e d
- $T(1)=d$

Ipotizziamo la soluzione $T(n) \leq kn$ per qualche k da determinare e sostituiamo:

- nel caso base: $d \leq k$
- nel caso generale:

$$T(n) \leq 2k n/2 + c = kn + c \text{ che non è MAI } \leq kn \dots$$

Metodo di sostituzione (10)

segue Esempio.

- $T(n) = 2T(n/2) + c$
- $T(1) = d$

Vuol dire che l'ipotesi è errata? non sempre:

Ipotizziamo la nuova soluzione $T(n) \leq kn - h$ per qualche k ed h da determinare e sostituiamo:

- nel caso base: $d \leq k - h$
- nel caso generale: $T(n) \leq 2(kn/2 - h) + c = kn - 2h + c \leq kn - h$ se $h \geq c$.

Da qui deduciamo che $T(n) = O(n)$.

Poi dobbiamo dimostrare anche che $T(n) = \Omega(n)$.

FARE PER ESERCIZIO

Metodo principale (1)



Idea:

avere una “ricetta” meccanica per risolvere un’equazione di ricorrenza

Difficoltà:

funziona solo quando l’equazione è della forma

$$T(n) = aT(n/b) + f(n) \text{ con } T(1) = \theta(1)$$

Metodo principale (2)

Enunciato del teorema principale:

Dati $a \geq 1$, $b > 1$, una funzione asintoticamente positiva $f(n)$ ed un'equazione di ricorrenza di forma:

$$T(n) = aT(n/b) + f(n), T(1) = \Theta(1)$$

vale che:

- Se $f(n) = O(n^{\log_b a - \varepsilon})$ per qualche costante $\varepsilon > 0$
allora $T(n) = \Theta(n^{\log_b a})$
- Se $f(n) = \Theta(n^{\log_b a})$ allora $T(n) = \Theta(n^{\log_b a} \log n)$
- Se $f(n) = \Omega(n^{\log_b a + \varepsilon})$ per qualche costante $\varepsilon > 0$ e se $a f(n/b) \leq c f(n)$ per qualche costante $c < 1$ e per n sufficientemente grande,
allora $T(n) = \Theta(f(n))$.

Metodo principale (3)

Il teorema principale ci dice che, in ciascuno dei tre casi, vengono confrontati fra loro $f(n)$ e $n^{\log_b a}$ ed il costo computazionale è governato dal maggiore dei due:

- se (caso 1) il più grande dei due è $n^{\log_b a}$, allora il costo è $\Theta(n^{\log_b a})$;
- se (caso 3) il più grande dei due è $f(n)$, allora il costo è $\Theta(f(n))$;
- se (caso 2) sono uguali, allora si moltiplica $f(n)$ per un fattore logaritmico.

Metodo principale (4)

Si noti che “più grande” e “più piccolo” in questo contesto significa polinomialmente più grande (o più piccolo), data la presenza all’esponente di ε . In altre parole, $f(n)$ deve essere asintoticamente più grande (o più piccola) rispetto a $n^{\log_b a}$ di un fattore n^ε per qualche $\varepsilon > 0$.

Metodo principale (5)

In effetti fra i casi 1 e 2 vi è un intervallo in cui $f(n)$ è più piccola di $n^{\log_b a}$, ma non polinomialmente.

Analogamente, fra i casi 2 e 3 vi è un intervallo in cui $f(n)$ è più grande di $n^{\log_b a}$, ma non polinomialmente.

Metodo principale (6) – caso 1

Esempio.

$$T(n) = 9T(n/3) + \Theta(n)$$

$$T(1) = \Theta(1)$$

- $a = 9, b = 3$
- $f(n) = \Theta(n)$
- $n^{\log_b a} = n^{\log_3 9} = n^2$

Poiché $f(n) = \Theta(n^{\log_3 9 - \varepsilon})$ con $\varepsilon = 1$, siamo nel caso 1,
per cui: $T(n) = \Theta(n^{\log_b a}) = \Theta(n^2)$.

Metodo principale (7) – caso 2

Esempio.

$$T(n) = T\left(\frac{2}{3}n\right) + \Theta(1)$$

$$T(1) = \Theta(1)$$

- $a = 1, b = 3/2$
- $f(n) = \Theta(1)$
- $n^{\log_b a} = n^{\log_{3/2} 1} = n^0 = 1$

Poiché $f(n) = \Theta(n^{\log_b a})$ siamo nel caso 2, per cui:

$$T(n) = \Theta(n^{\log_b a} \log n) = \Theta(\log n).$$

Metodo principale (8) – caso 3

Esempio.

$$T(n) = 3T(n/4) + \Theta(n \log n)$$

$$T(1) = \Theta(1)$$

- $a = 3, b = 4$
- $f(n) = \Theta(n \log n)$
- $n^{\log_b a} = n^{\log_4 3} \approx n^{0,7}$

Poiché $f(n) = \Omega(n^{\log_4 3 + \varepsilon})$, ad esempio con $\varepsilon = 0,2$, siamo nel caso 3 se possiamo dimostrare che $3\frac{n}{4} \log \frac{n}{4} \leq c n \log n$, per qualche $c < 1$ ed n abbastanza grande.

Ponendo $c = \frac{3}{4}$ otteniamo $3\frac{n}{4} \log \frac{n}{4} \leq \frac{3}{4} n \log n$

che è vera, quindi: $T(n) = \Theta(n \log n)$.

Metodo principale (9)

Esempio.

$$T(n) = 2T(n/2) + \Theta(n \log n)$$

$$T(1) = \Theta(1)$$

- $a = 2, b = 2$
- $f(n) = \Theta(n \log n)$
- $n^{\log_b a} = n^{\log_2 2} = n$

Ora, $f(n) = \Theta(n \log n)$ è asintoticamente più grande di $n^{\log_b a} = n$, ma non polinomialmente più grande. Infatti, $\log n$ è asintoticamente minore di n^ϵ per qualunque valore di $\epsilon > 0$. Di conseguenza non possiamo applicare il metodo del teorema principale.

Esercizi per casa



Esercizi

Calcolare la soluzione delle seguenti equazioni di ricorrenza con tutti e quattro i metodi ove possibile:

- $T(n)=2T(n/2)+\Theta(n)$ $T(1)=\Theta(1)$
- $T(n)=3T(n/2)+\Theta(n)$ $T(1)=\Theta(1)$
- $T(n)=3T(n/4)+\Theta(n)$ $T(1)=\Theta(1)$
- $T(n)=2T(n/2)+\Theta(n)$ $T(1)=\Theta(1)$
- $T(n)=4T(n/2)+\Theta(n)$ $T(1)=\Theta(1)$
- $T(n)=2T(n/2)+\Theta(n^3)$ $T(1)=\Theta(1)$
- $T(n)=16T(n/4)+\Theta(n^2)$ $T(1)=\Theta(1)$
- $T(n)=T(n-1)+\Theta(n)$ $T(1)=\Theta(1)$
- $T(n)=3T(n/2)+\Theta(n \log n)$ $T(1)=\Theta(1)$